
UNIT 8 DATABASE MANAGEMENT ISSUES IN MOBILE COMPUTING

8.0 Introduction

8.1 Objectives

8.2 Mobile Device Database Management

8.2.1 Brief Details About Database Management System

8.3 Mobile Device Data Store Methods

8.4 Client Server Computing With Adaptation For Mobile Computing

8.5 Adaptation Software For Mobile Computing

8.6 Summary

8.7 Solutions/Answers

8.8 Further Readings

8.0 INTRODUCTION

We store the business data in a database management system. With the explosion of various devices and increase in users' access to the Internet, the data volume has exploded in the recent past. The application stores the core application data, configuration data, transaction data and other business data in the database system. Few applications also store the log data and audit data in the database.

As mobile has become the primary gateway for Internet access, the importance of mobile database has grown exponentially. Modern application architectures use offline-first approach where mobile applications store the data in the local database on the device for providing enhanced user experience during network issues. Mobile applications also use device database for storing the user profile data, cached data, configuration data and other details.

Normally the data is stored in files or in memory cache or in the database. For instance, a mobile device stores the pictures in the local device folder in the file format. As mobile devices are not always connected to the network we need to ensure that data is available in the offline mode.

In this unit we shall examine the key database management issues in the mobile computing

8.1 OBJECTIVES

After going through this unit, you should be able to

- understand key concepts of database,
- understand various types of databases,
- understand the issues with mobile data management,
- understand various data store methods for mobile,
- understand the client server computing for mobile computing, and
- understand adaptation software for mobile computing.

8.2 MOBILE DEVICE DATABASE MANAGEMENT

In this section we discuss the brief details of database management for the mobile computing.

8.2.1 Brief Details about database management system

A database management system is used for organizing information that can be easily persisted, queried and managed.

Based on the nature of the data, we can leverage various kinds of databases. Data is organized in structured way to easily persist and retrieve in relational database systems. In No-SQL databases we mainly store the key-value pairs, or documents or graph-based data or unstructured data.

Each data record is organized as a database row. We store the entity data in a database table. The properties of the entity are managed as attributes. Lets consider a simple example of representing the student entity in the database table

StudentId	StudentName	StudentPhone	StudentAddress
123	Student1	1231231234	Bangalore
456	Student2	1234123456	Delhi

In the above example we have depicted various attributes of student entity such as studentid, studentname, studentphone and studentaddress. We also have two rows representing two data records.

We can perform various operations on the data stored in the database. We have given the key operations:

- Query operation – we can query the structured data from the database table. We can also filter the data using a conditional operator.
- Insert operation – We can insert the data into the specific table using the insert operation.
- Delete operation – We can delete a row(s) from the database based on the condition.

A database transaction consists of series of database operations (for example insert operation followed by delete operation). The database ensures the consistency and data integrity of the transaction.

We can establish a foreign key relationship with other tables when two entities are related. For instance, lets consider a course table as below

CourseId	CourseName
1	Computer Science
2	Electronics & Communication

If we need to associate the students to the courses, we can introduce a foreign key to the Students table as shown below

StudentId	StudentName	StudentPhone	StudentAddress	CourseId
123	Student1	1231231234	Bangalore	1
456	Student2	1234123456	Delhi	2

We have now introduced the CourseId foreign key to the student table. We can associate the student to a course using the foreign key. We can filter the data to using the foreign keys; for instance in the students table we can query all the students who have taken up Computer Science course by joining the Students table with Courses table and matching the CourseId.

Types of databases

There are various kinds of databases. We can use the most appropriate database based on the use case. Given below are the some of the most popular databases

- Relational database: The data is stored in tables which are related to each other. For structured data use cases such as financial data we use relational database. MySQL, Oracle, MS SQL Server are some of the popular relational database systems.
- No SQL database: The database schema/model is flexible in No SQL database. We can store the session data, shopping card data in No SQL database. MongoDB, DocumentDB are few examples of No SQL database engines.
- XML Database: We use the XML tags as the keys in the XML database. We can use the XML database to manage the key value pair use cases.

Mobile database management systems

SQL Anywhere mobile device database, IBM DB2 Everyplace, Oracle 9i Lite, Microsoft SQL compact, SQLite are some of the popular mobile device databases. All mobile database systems use small memory footprint due to the limited resource availability on mobile devices. Mobile devices can store the application configuration, user preferences, user profile and other offline data in the local database.

Explicit business logic and Implicit business logic

The mobile apps use APIs to interact with the database. When the business logic is explicitly specified in the API, it is termed as explicit business logic. For instance, in an ecommerce application when we would like to display a product details page for a specific product we send the product id as a filter parameter such as follows:

If product_id=1234 and geography="India" then Get_product_details

The above logic filters the data explicitly based on the product_id and geography.

In an implicit business logic, the structure of the database is used to retrieve the data from the database. For instance when you search for a contact name in the

contact list, the mobile app automatically shows the phone number associated with the contact. The association of phone number for the contact is implicit.

☞ Check Your Progress 1

1. Key value pairs are stored in ____ database
2. Series of database operations that ensure consistency is called _____
3. When the business logic is specified in the API it is called _____ business logic
4. In an _____ business logic, the structure of the database is used to retrieve the data from the database

8.3 Mobile Device Data Store Methods

Mobile devices have limited storage capacity. We cannot store large quantities of data on mobile devices. For dynamic mobile apps we need to retrieve the data from the server. If the mobile device tries to retrieve large quantity of data for every screen refresh, it impacts the end user experience.

Hence to balance the dynamic nature of the data and the mobile app performance, the mobile app needs to store the data locally on the device. Storing the frequently used data locally on the device is referred to as caching or hoarding.

Normally the mobile device caches the data fetched from the remote server. For subsequent calls, the mobile device uses the locally cached data; the mobile app uses the locally cached data during the absence of network. The mobile device cache is refreshed when the mobile is connected to the network. For instance, a learning mobile app that teaches the vocabulary caches the most frequently used words in the local device cache. When the device connects to the Internet it refreshes the data.

We shall look main architecture patterns for mobile device data store methods.

Single Tier Architecture

In this architecture model, the mobile app directly interacts with the embedded database on the mobile device. We could use embedded mobile database such as SQLite which stores the frequently access mobile app data. The embedded mobile database stores the core application data such as business data, location data, user data that enables the mobile app to be operated in the offline mode as well.

Figure 1 depicts the single tier architecture database

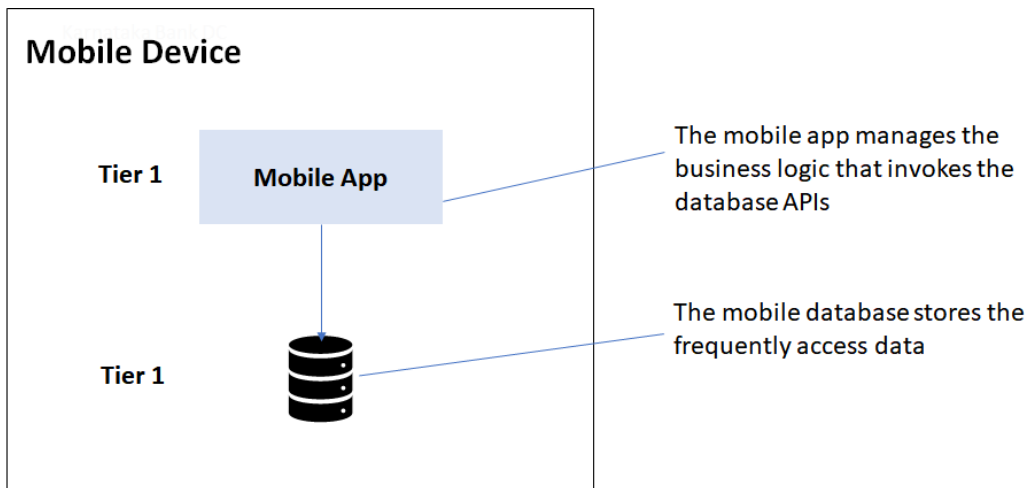


Figure 1 Single Tier Mobile database architecture

In single tier mobile database architecture, the entire mobile app data is hoarded within mobile device.

Multi-Tier Architecture

In a typical multi-tier architecture, the mobile app uses the data stored in the remote database through APIs. Figure 2 depicts a multi-tier architecture. The mobile app in tier 1 invokes the business services in tier 2 which gets the data from the database in tier 3.

The mobile app can optionally cache the retrieved data locally in a cache or a database. This reduces the overall latency in invoking the data from the remote database. We have depicted the local caching architecture in Figure 3.

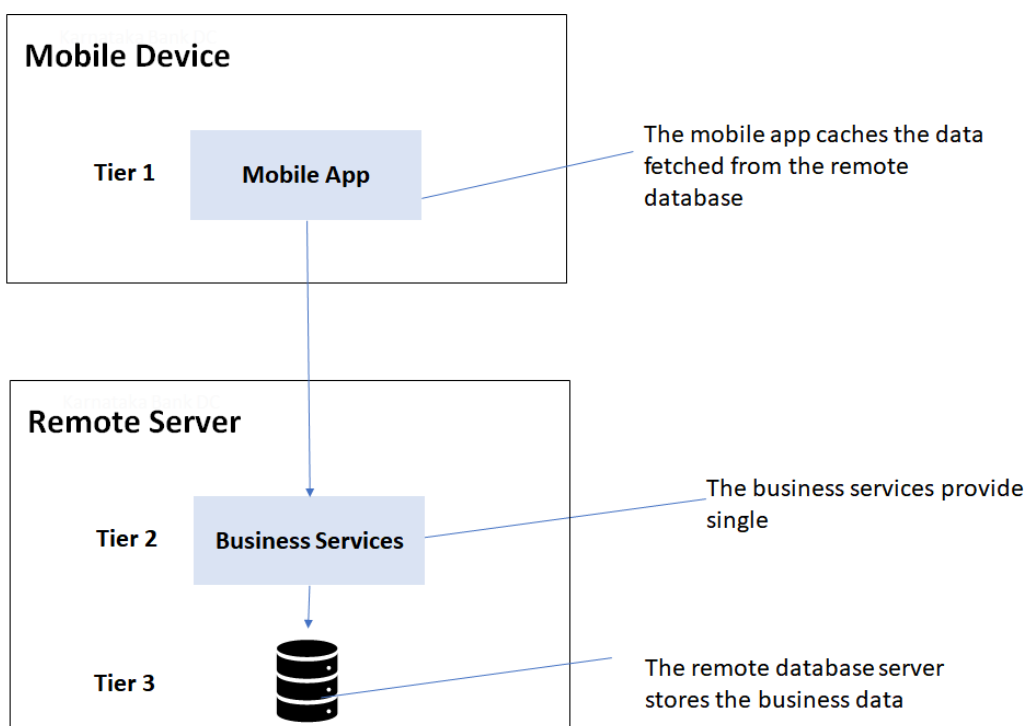


Figure 2 Multi-tier Mobile Database Architecture

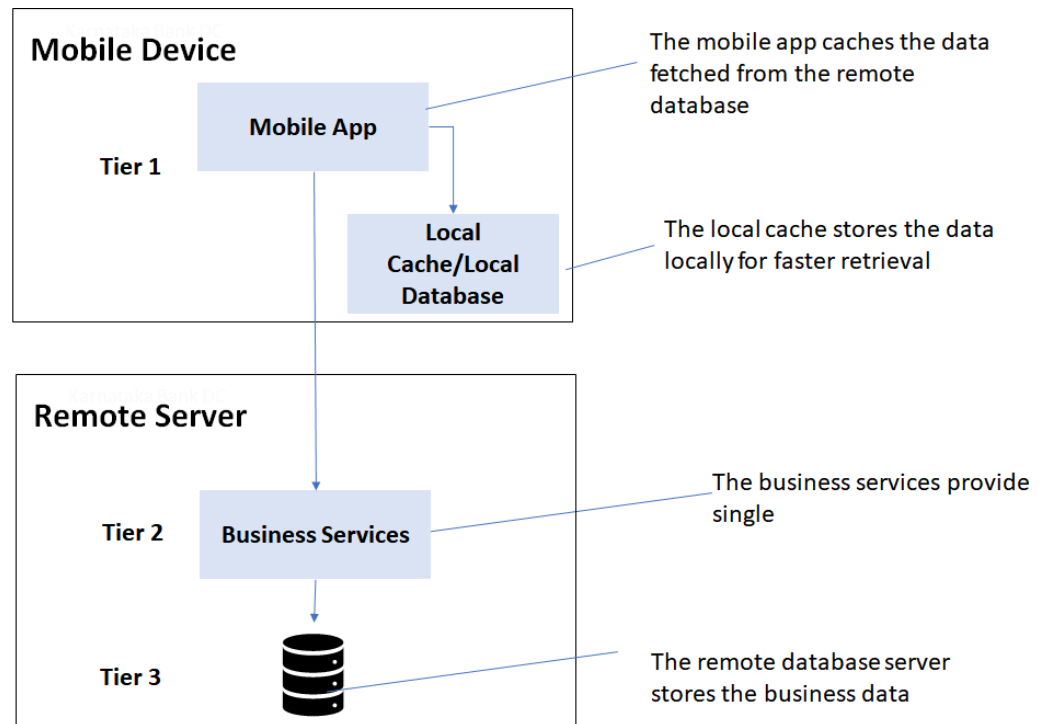


Figure 3 Multi-tier Mobile Database Architecture With Local Caching

The local cache in Figure 3 periodically synchronizes the data from the remote server so that the mobile app can invoke the data faster.

We have depicted a n-tier architecture in Figure 4. We have depicted various SDKs that will be integrated with the mobile app. We also have depicted various functional modules such as login module, products module, home screen module and such.

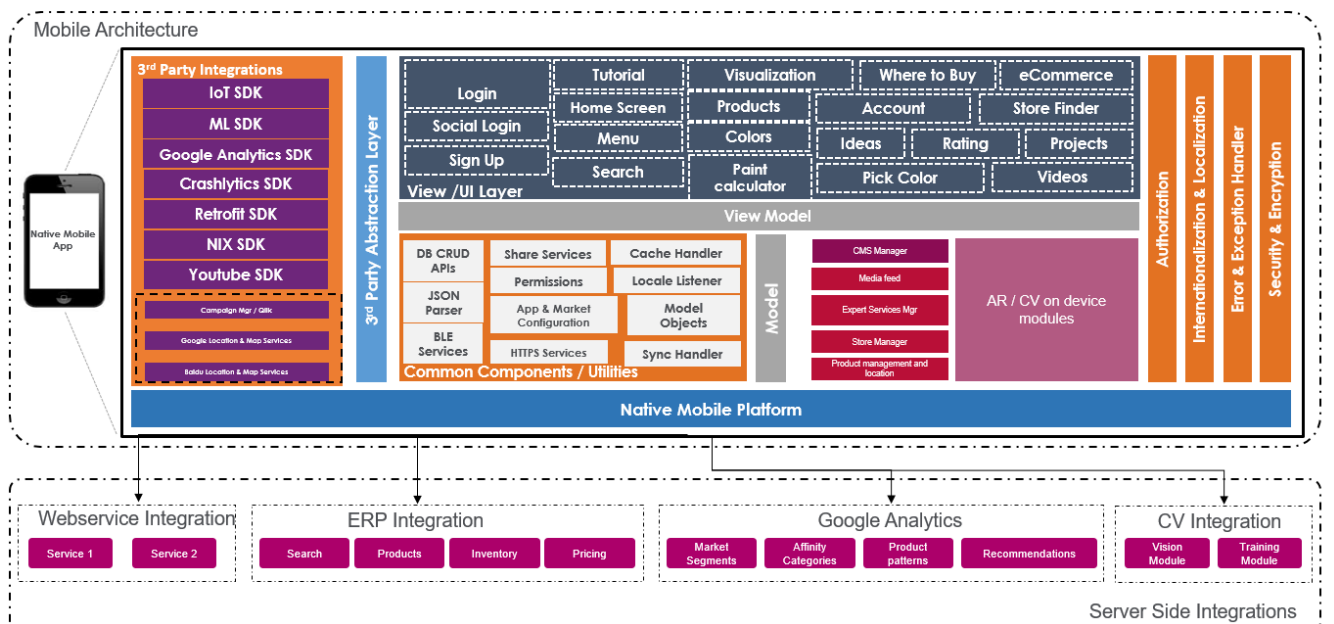


Figure 4 Multi-tier Architecture for an enterprise mobile app

The main integrations for the enterprise mobile app are ERP, Google analytics, web services and OpenCV. All these applications are integrated through APIs.

Caching Patterns

In the multi-tier architecture the mobile app caches the data to reduce the overall latency. The general thumb rule for the data to be cached is the access probability of the cached data. If the data is regularly accessed (for instance user detail information or static information that is displayed on each mobile screen) then it is an ideal cache candidate.

Broadly there are two methods to populate the cache. In the first method the mobile app pulls the required data from the remote server and caches that data. The mobile app can pull the most frequently used data to reduce the overall latency. The second one is the push method where the server pushes the data. The push is mainly used by the server to synchronize the server data with the client data. Among the push and pull methods, the pull method is optimal as only the required data is pulled when it is required.

We have depicted the pull pattern of caching in Figure 5 where the mobile client pulls the cached content from the systems.

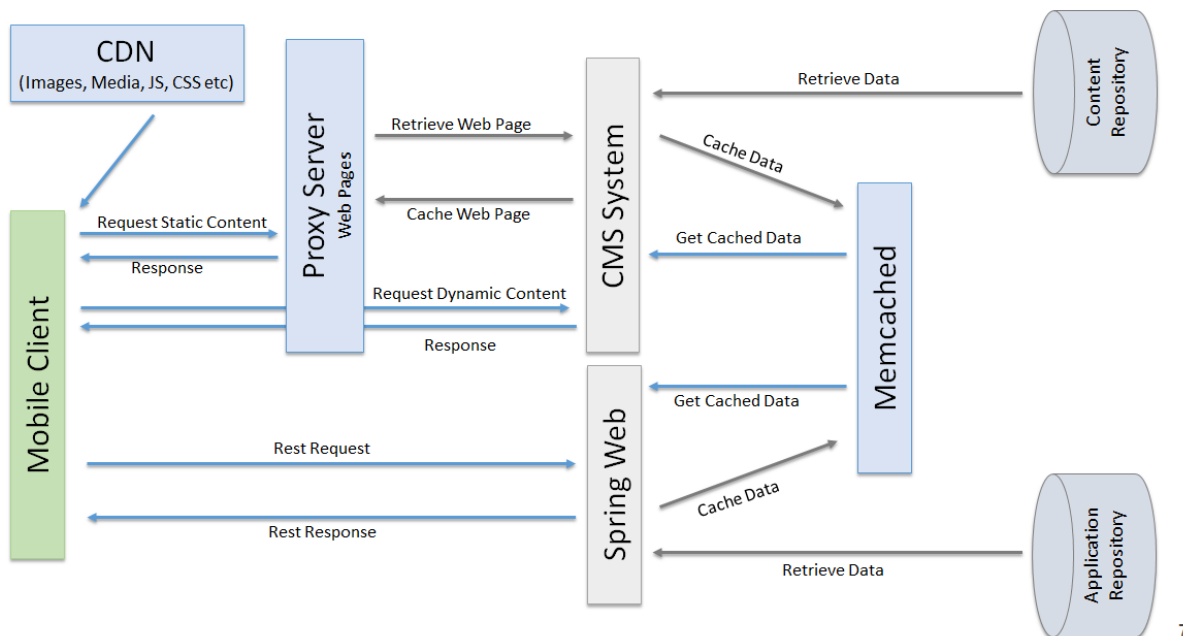


Figure 5 Pull pattern of caching

Another caching technique is the pre-fetch wherein the mobile app prefetches the data in advance in anticipation of its need. For instance if the mobile user has visited the product list page, the mobile app can prefetch the product details in anticipation of mobile user visiting the product details page.

When the mobile app does not find the required data in the local cache it is called “cache miss” and the mobile app has to go back to the server to get the required data.

We need to use appropriate cache invalidation mechanism to ensure that the mobile client gets the accurate data. There are broadly four cache invalidation methods:

- Stateless asynchronous in which a broadcast message invalidates all cached objects of all clients of the server
- Stateless synchronous in which the server broadcasts the cache invalidation method when the data is change or modified.
- Stateful asynchronous where the server broadcasts the cache invalidation message only for the specific clients who are impacted. The server does not keep track of the client data
- Stateful synchronous where the server keeps track of the client data and broadcasts cache invalidation message to the specific client when the data is changed.

8.4 Client Server Computing with Adaptation for Mobile Computing

In the evolution of the enterprise architectures we initially had mainframes. In the next stage we had client server architecture. In client-server architecture, the heavy computing and memory resources are deployed in the server. The server handles the core business computing and data processing. The clients have lesser resource capacity and they simply connect to the servers to retrieve the data or submit the job. Terminal applications and database clients, thick clients mainly used the client-server architecture.

In the mobile computing, the mobile apps are equivalent to the client applications. As mobile devices have limited resources such as memory, storage they retrieve the data from the servers. We have depicted a 4-tier architecture in Figure 6 where mobile is the client application

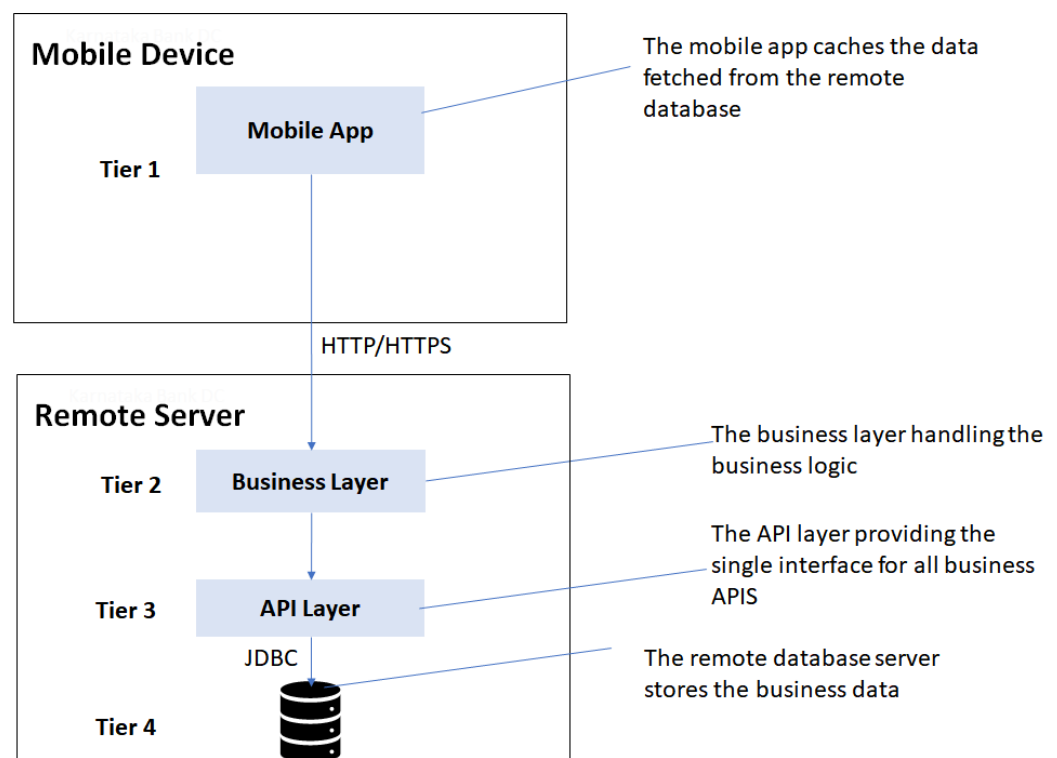


Figure 6 Four-tier mobile computing architecture

The 4-tier architecture depicted in Figure 4, the mobile app interacts with the business layer through HTTP or HTTPS. The mobile app invokes the business services through modern architecture patterns such as REST over HTTP. The business layer centralizes the business logic processing and uses the appropriate APIs in the API layer. The API layer is the single interface for all backend integrations. The API layer provides APIs to retrieve the data from enterprise database, from web services and from ERP services. To retrieve the data from enterprise database, the API layer uses JDBC calls. The enterprise data is stored in the centralized enterprise database. In the 4-tier architecture, the mobile app can also cache or hoard the data locally to improve the performance. The local cache or database on the mobile device synchronizes with the remote server using synchronization server.

In a typical n-tier architecture each layer has distinct responsibilities and handle single concern. This architecture design is called “separation of concerns” and “Single responsibility principle” These principles helps in building the decoupled architecture where each layer can be scaled independently.

8.5 Adaptation Software for Mobile Computing

The mobile device uses multiple interface such as email, contacts, calendar and others. As the data format sent by the synchronization server is different from the data structure expected by the database, we need an adaptor that converts the data format of the synchronization server to that of the database.

The adaptors convert the data from the standard format into the format needed for the APIs and interfaces.

We have depicted the adaptation software in Figure 7.

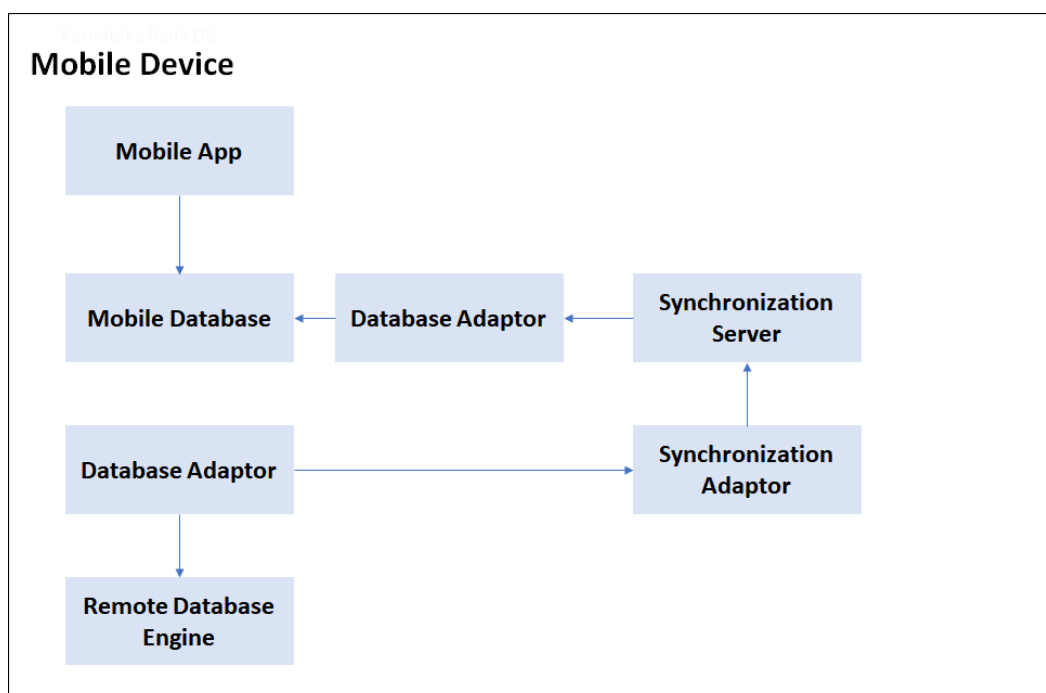


Figure 7 Adaptors at mobile device

As shown in Figure 6, we mainly need two adaptors. The first adaptor that converts the data received from the synchronization server to the mobile database. This database adaptor is present in the mobile end.

The second adaptor is at the remote server which converts the data from the remote database to the synchronization data.

☞ Check Your Progress 2

1. In single tier architecture, mobile app interacts with the embedded database ____
2. In multi tier architecture, the mobile app invokes the data stored ____ through APIs
3. In ____ method of caching, the server sends data to be cached to the client
4. The caching method in which the client requests for the data to be cached is called ____ method
5. In ____ method of cache invalidation all the clients receive a cache invalidation broadcast.

8.6 SUMMARY

In this unit, we started discussing the core concepts of the database. In SQL databases the data is managed in a structured schema. Key values are stored in NoSQL databases. In single tier architecture for mobile computing, the mobile app directly retrieves the data from the embedded database. In the multi-tier architecture, the mobile app retrieves the data stored in the remote database through APIs. Caching can be mainly done using push method wherein the server pushes the data to be cached to the client and pull method wherein the client pulls the data to be cached from the server. In a typical n-tier architecture in mobile computing the mobile app retrieves the data through APIs.

8.7 SOLUTIONS/ANSWERS

Check Your Progress 1

1. NoSQL
2. transaction
3. explicit
4. implicit

Check Your Progress 2

1. directly
2. remotely
3. push
4. pull
5. stateless asynchronous

8.8 FURTHER READINGS

References

Mobile Computing 3rd Edition by Raj Kamal -
<https://www.amazon.in/Mobile-Computing-Raj-Kamal/dp/0199455414>